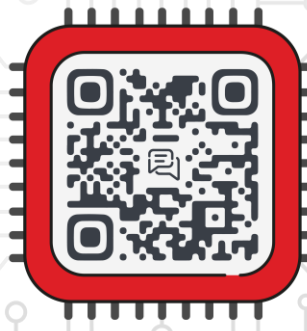




نظري

2

5

600 sp

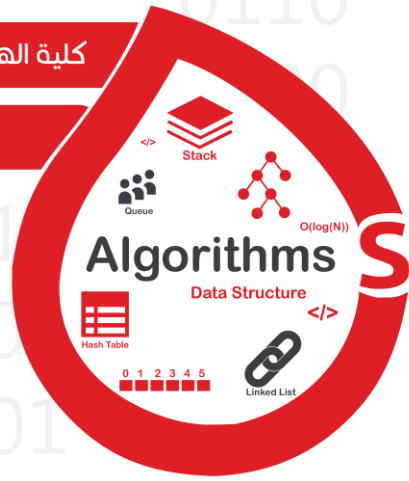
كلية الهندسة المعلوماتية

السنة الثانية

التعقيد الزمني

Time Complexity

د. أبي صندوق



RB Informatcs; 27/10/2024

خوارزميات 1

مقدمة :

الهدف من المادة هو إيجاد طرق تفكير مختلفة ، نفس المسألة يمكن حلها بأكثر من طريقة ، فكيف لنا أن نختار الطريقة الأفضل ؟

التعقيد الزمني

هو أحد الأدوات التي ستساعدنا باختيار طريقة الحل الأفضل

أي كم وحدة زمن ستستهلك الخوارزمية عندما تعمل.

قد يختلف الزمن الذي ستستهلكه الخوارزمية من حاسوب إلى آخر وحسب الشخص الذي كتبها وطبيعة المدخلات والكثير من العوامل الأخرى التي تؤثر على الزمن الحقيقي لتنفيذ هذه الخوارزمية ، لكننا سنهتم بما هو أعم من هذا وهو درجة

الخوارزمية

⚡ درجة الخوارزمية :

أي : إذا كانت الخوارزمية من رتبة معينة فإنها تقريباً ستعمل بنفس السرعة على جميع الحواسيب (أسرع أو أبطأ قليلاً)

▪ إذا كانت الخوارزمية من رتبة أعقد (أعلى) ستأخذ زمن أكثر وبالتالي ستكون أسوأ بغض النظر عن الحاسوب .

▪ إذا كانت الرتبة أقل ستكون أسرع بغض النظر عن الحاسوب .

نقدّر التعقيد الزمني بعدد العمليات التي تحتاجها الخوارزمية .

■ ملاحظة:

بما أن الحواسيب متقاربة من بعضها فإن الخوارزميات التي يكون عدد عملياتها أقل سوف تكون أسرع بغض النظر عن الحاسوب المطبق عليه الخوارزمية (لأن عدد العمليات بالخوارزمية نفسها لا يختلف من حاسوب لآخر).

مثال: ((الكود بلغة JAVA))

حساب مجموع عناصر مصفوفة

A

Cost	Time
—	—
2C	1
2C	N+1
2C	N
1C	1

```
int SumArray(int []A) {
```

```
1) int sum = 0;
2) for (int i = 0; i < A.length; i++)
3)     sum = sum + A[i];
4)     return sum;
}
```

الزمن اللازم لتنفيذ الكود هو **مجموع** الزمن اللازم لتنفيذ كل عملية .

Cost : هو زمن ثابت للعملية ويختلف من حاسوب لآخر (التكلفة).

Time : هو عدد مرات الوصول / تكرار هذه العملية .

طريقة الحل:1. نحتاج عملية لحجز مكان في الذاكرة للمتغير sum وعملية إسناد للصفر $\leftarrow 2C$.2. نحتاج عملية للمقارنة ($i < A.length$) وعملية $i++$ $\leftarrow 2C$.

وسنكرر الحلقة N+1 (في المرة الأخيرة لن يدخل إلى داخل الحلقة ولكن سيقوم بفحص الشرط فقط) .

ملاحظة

■ بالنسبة ل ($int i = 0$) فهي تُكرر مرة واحدة فقط لذا إن أردنا الدقة يمكن فصلها بتعليمة منفصلة هكذا .

	Cost	Time
int i=0;	2C	1

3. نحتاج عملية جمع و عملية إسناد ($2C$) وستكرر بعدد عناصر A أي (N) مرة4. عملية واحدة فقط ($1C$) ولمرة واحدة (1) مرة .**حساب الزمن اللازم لتنفيذ الكود :**

$$T(N) = 2C + 2C(N+1) + 2C(N) + C$$

$$= 4N \cdot C + 5C$$

$$= (4N+5) C$$

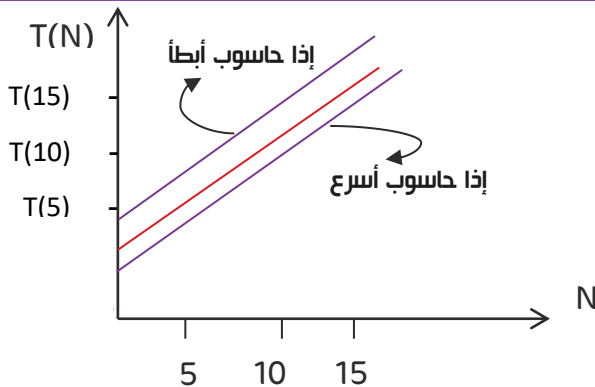
وحدة الزمن
sec

بما أنه مضروب ب C فله واحدة زمنية

■ بدون الثابت الزمني سوف تكون $4n+5$ ترمز إلى عدد العمليات .

ملاحظة :

- لا يتأثر الزمن بحجم (قيمة) عناصر المصفوفة سواء كانوا آحاد أم ملايين لأن العمليات ستبقى نفسها مهما كانت قيمة العناصر لكنه يتأثر بعدد العناصر .



- إذا كان عدد العناصر خمسة ، $N=5$:
 $T(5) = (4 * 5 + 5) C = 25 C$
- إذا كان عدد العناصر عشرة ، $N=10$:
 $T(10) = 45 C$
- إذا كان عدد العناصر خمسة عشر ، $N=15$:
 $T(15) = 65 C$

كلما ازداد N عدد العناصر بمقدار واحد ، يزداد الزمن 4 وحدات زمنية (4 عمليات) بشكل خطي .

نستنتج من الخط البياني أنه :

سيبقى التابع $T(N)$ خطياً حتى لو زاد الثابت الزمني أو نقص .

ملاحظة :

- لماذا لم يبدأ المستقيم من الصفر ؟
لأنه عند $N=0$ سوف يكون $T(0)=5$.
- الاختلاف بين الحاسوب الأسرع والأبطأ في قيمة C

مثال:

- حساب مجموع عناصر مصفوفة من بدايتها إلى عنصر معين M :

حتى يكون الكود صحيح أكثر نتأكد أن M هي أقل من $A.length$

	Cost	Time
<code>int SumArray(int []A , int M) {</code>	—	—
1) <code>int sum = 0;</code>	$2C$	1
2) <code>for (int i = 0; i <= M; i++)</code>	$2C$	$M+1$
3) <code>sum = sum + A[i];</code>	$2C$	M
4) <code>return sum;</code>	$1C$	1
}		

حساب الزمن اللازم لتنفيذ الكود :

$$T(M) = 2C + (M + 1) * 2C + (M) * 2C + C$$

$$= (4M+5) C \quad \text{sec}$$

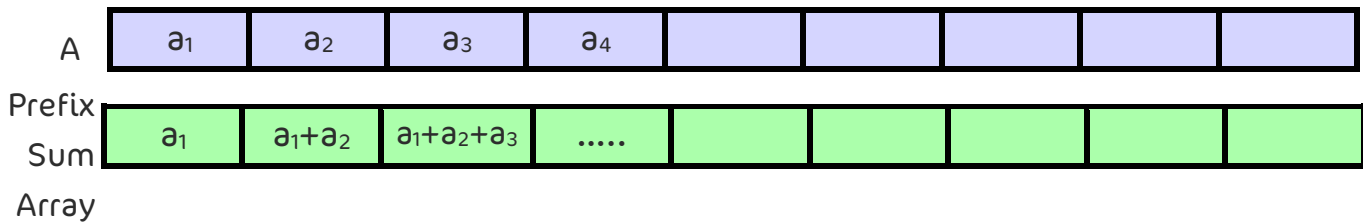
■ ملاحظة :

الذي يهمنا في التابع الزمني في هذه الحالة هو عدد العناصر التي نريد جمعها (M) وليس عدد عناصر المصفوفة.

■ مثال :

باستعمال التابع السابق اكتب كود يحسب ال Prefix Sum

■ Prefix Sum :



ما هو ال prefix sum ؟

تخيل لديك قائمة بالأرقام، مثلاً: [3, 1, 4, 1, 5].

ال prefix sum هو قائمة أخرى بنفس الطول، ولكن كل عنصر فيها يمثل مجموع كل العناصر التي تسبقه في القائمة الأصلية بالإضافة إلى نفسه. أي كل عنصر [j] يحوي مجموع العناصر من [0] إلى [j].

	Cost	Time
<code>int [] PrefixSum (int []A){</code>	—	—
<code>1) int B[] = new int [A.length];</code>	C	1
<code>2) for (int i =0; i< A.length; i++)</code>	2C	N+1
<code>3) B[i]=SumArray(A,i);</code>	T(i)	N
<code>4) return B;</code>	1C	1
<code>}</code>		

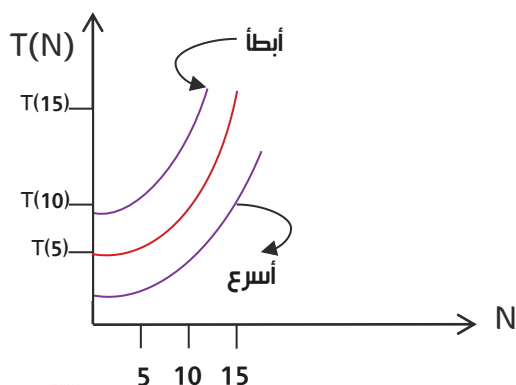
حساب الزمن اللازم لتنفيذ الكود :

■ للتخلص من ا:

$$T(N) = C + (N+1)*2C + T(i)*N + C$$

$$T(N) = C + (N+1)*2C + (2N^2 + 7N)C + C$$

$$(2N^2 + 9N + 4) C \text{ sec}$$



$$T(i) * N = \sum_{i=0}^N T(i) = \sum_{i=0}^N (4i + 5)c = (4 \sum_{i=0}^N i + 5N) c = \left(4 \frac{N*(N+1)}{2} + 5N \right) c = (2N^2 + 7N)c$$

قانون مجموع الأعداد الطبيعية من 0 إلى n

$$T(5) = 99c$$

$$T(10) = 294c$$

$$T(15) = 589c$$

التابع قطع مكافئ (تربيعي)

لذا الجزء المهم في التعقيد الزمني هو الدرجة (أن التابع تربيعي) بغض النظر عن الأعداد المضروب بها أو الحاسوب إن كان بطيئاً أو سريعاً .

يمكن حل المسألة السابقة بتعقيد خطي .

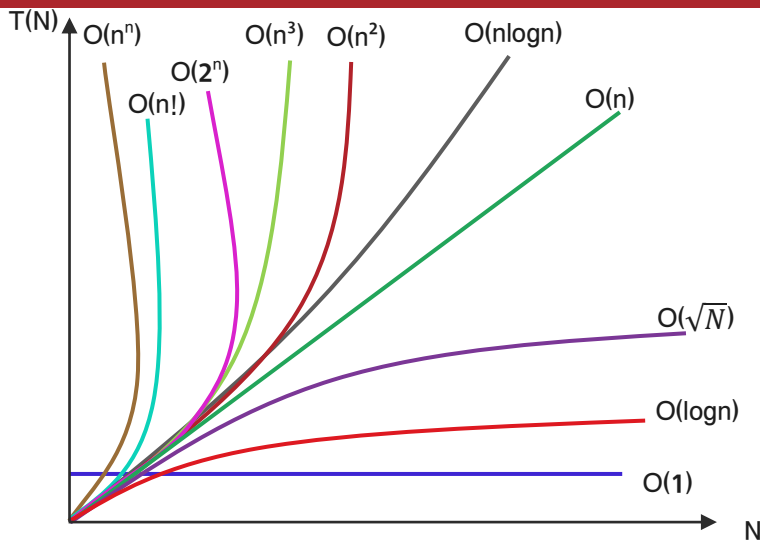
نستنتج أن :

ما يهم في حساب التعقيد الزمني هو رتبة (درجة) الخوارزمية وفي النهاية مهما كانت العوامل المؤثرة (مضروب برقم معين أو مضاف إلى رقم معين سواء كانوا أرقاماً كبيرة أو صغيرة أو) فإنه حدّ معين حتماً سيبقى التابع التربيعي أسوأ من الخطي .

■ ملاحظة :

في حال كان عدد العناصر قليل فالزمن سيكون قليلاً سواء كان التعقيد خطياً أم تربيعياً ، وبالتالي نختار الخوارزمية حسب ال trade-off فقد نختار مثلاً الخوارزمية الأسرع بالكتابة .

أنواع التوابع



O: Order of N

- O (1) (1)
- O (log N) (2)
- O(√N) (3)
- O (N) (4)
- O (N log N) (5)
- O (N²) (6)
- O (2ⁿ) (7)
- O (N!) (8)

Knowledge is power....

THE END

